



National University
of computer and emerging sciences

Database Systems Lab (CL-2005)

Semester: Spring 2026

Section: BDS-4B

Course Instructor: Mr. Jahan Zaib

LAB # 05

Aggregate Functions & DML Queries

Objectives:

After completing this lab, students will be able to:

- Understand the concept and purpose of aggregate functions in Oracle SQL.
- Apply aggregate functions such as AVG, COUNT, SUM, MIN, and MAX to perform calculations on grouped data.
- Use the GROUP BY clause to organize data into summary rows based on one or more columns.
- Implement the HAVING clause to filter grouped results using aggregate conditions.
- Differentiate between the WHERE and HAVING clauses in query execution.
- Write and execute DML (Data Manipulation Language) statements including INSERT, UPDATE, and DELETE.
- Insert single and multiple records into tables using Oracle SQL syntax.
- Modify existing records using the UPDATE statement with appropriate conditions.
- Remove specific or all records from a table using the DELETE statement.
- Develop efficient and logically correct SQL queries following Oracle's execution order.

Aggregate Functions

An **aggregate function** performs a calculation on a set of rows and returns a single scalar value. Aggregate functions are commonly used with the GROUP BY and HAVING clauses in a SELECT statement. **WHERE** clause comes before GROUP BY clause.

Important Rules in Oracle SQL

- The WHERE clause comes before the GROUP BY clause.
- The HAVING clause comes after the GROUP BY clause.
- Column aliases cannot be used in the GROUP BY clause in Oracle.
- Any column in the SELECT list that is not inside an aggregate function must appear in the GROUP BY clause.
- The GROUP BY clause can include more than one column.

Most Common Aggregate Functions

- **AVG** – Calculates the average of non-null values.
- **COUNT** – Counts rows.
- **MIN** – Returns the minimum value.
- **MAX** – Returns the maximum value.
- **SUM** – Calculates the total of non-null values.

NULL Handling in Oracle

- AVG, SUM, MIN, and MAX ignore NULL values.
- COUNT(expression) ignores NULL values.
- COUNT(*) counts all rows, including rows containing NULL values.

Syntax of Aggregate Functions

`aggregate_function (DISTINCT | ALL expression)`

- DISTINCT → Considers only unique values.
- ALL → Considers all values (default option).
- If neither is specified, ALL is used automatically.

GROUP BY Statement

The GROUP BY statement groups rows that have the same values into summary rows. It is commonly used with aggregate functions to perform calculations on grouped data.

For example: Finding the number of employees in each department.

Syntax:

```
SELECT column_name(s)
FROM table_name
WHERE condition
GROUP BY column_name(s)
ORDER BY column_name(s);
```

Important Oracle Rule

Every column in the SELECT clause that is not part of an aggregate function must be included in the GROUP BY clause.

Example (Correct in Oracle):

```
SELECT department_id, SUM(salary)
```

```
FROM employees  
GROUP BY department_id;
```

Example (Invalid in Oracle):

```
SELECT department_id, salary  
FROM employees  
GROUP BY department_id;
```

(Invalid because salary is neither grouped nor aggregated.)

GROUP BY with Multiple Columns

```
GROUP BY department_id, job_id;
```

DISTINCT vs GROUP BY

Distinct is used to find unique/distinct records where as a group by is used to group a selected set of rows into summary rows by one or more columns or an expression.

The group by can also be used to find distinct values as shown in below query.

```
SELECT DEPARTMENT_ID  
FROM EMPLOYEES  
GROUP BY DEPARTMENT_ID;
```

OR

```
SELECT DISTINCT DEPARTMENT_ID  
FROM EMPLOYEES ;
```

HAVING clause

The HAVING clause is used to filter grouped results. It was introduced because the WHERE clause cannot be used with aggregate functions.

Syntax:

```
SELECT column_name(s)  
FROM table_name  
WHERE condition  
GROUP BY column_name(s)
```

HAVING *condition*

ORDER BY *column_name(s)*;

Example:

```
SELECT COUNT(CustomerID), Country
FROM Customers
GROUP BY Country
HAVING COUNT(CustomerID) > 5
ORDER BY COUNT(CustomerID) DESC;
```

The above SQL statement lists the number of customers in each country, sorted high to low (Only include countries with more than 5 customers):

Order of Execution of an Oracle SQL Query

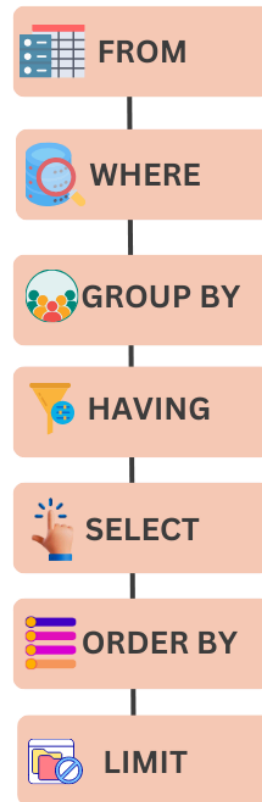
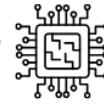
In Oracle Corporation SQL, the execution of a query does not follow the same order in which the clauses are written. Although we write a query starting with the SELECT clause, Oracle processes the query in a logical sequence. First, the FROM clause is executed to identify and retrieve the source tables. Next, the WHERE clause filters rows based on specified conditions. After that, the GROUP BY clause groups the filtered rows, and aggregate functions are applied. The HAVING clause then filters the grouped results. The SELECT clause is processed to retrieve the required columns, and finally, the ORDER BY clause sorts the result set. Understanding this logical execution order helps in writing correct and optimized SQL queries in Oracle.



Query Structure



Execution Order



Pipeline to I

Figure 1: Order of SQL Execution Query

DML (Data Manipulation Language)

DML (Data Manipulation Language) consists of SQL commands used to manipulate data stored in database tables. These commands allow users to insert, modify, and delete records.

Examples of DML Commands in Oracle

- **INSERT** – Adds new records into a table.
- **UPDATE** – Modifies existing records in a table.
- **DELETE** – Removes records from a table.

INSERT Statement (Oracle)

The INSERT statement is used to add new rows into a table in Oracle.

Records can be inserted:

- One row at a time
- Multiple rows using INSERT ALL
- Using data from another table (INSERT INTO SELECT)

Creating Tables

```
CREATE TABLE tasks (  
    task_id NUMBER GENERATED BY DEFAULT AS IDENTITY,  
    title VARCHAR2(255) NOT NULL UNIQUE,  
    start_date DATE,  
    due_date DATE,  
    priority NUMBER(1) DEFAULT 3 NOT NULL,  
    description CLOB,  
    CONSTRAINT pk_task PRIMARY KEY (task_id)  
);
```

```
CREATE TABLE employee (  
    empNo VARCHAR2(8) PRIMARY KEY,  
    empName VARCHAR2(30) NOT NULL,
```

```
department VARCHAR2(30) NOT NULL,  
email VARCHAR2(30) NOT NULL,  
phone VARCHAR2(30)  
);
```

Note: Oracle uses IDENTITY instead of AUTO_INCREMENT and VARCHAR2 instead of VARCHAR.

Simple INSERT Example

```
INSERT INTO tasks  
(task_id, title, start_date,  
due_date, priority,  
description)
```

```
VALUES (123, 'Learn  
Oracle INSERT', DATE  
'2020-06-22', DATE  
'2020-06-27', 2, 'Oracle  
test example');
```

OR

```
INSERT INTO tasks
```

```
VALUES (124, 'Another Task', DATE '2020-06-22', DATE '2020-06-27', 2, 'Sample');
```

Insert Data into Specific Columns

```
INSERT INTO tasks (title,  
priority)  
VALUES ('Understanding  
DEFAULT keyword',  
DEFAULT);
```

If columns are not specified:

- Oracle inserts NULL where allowed
- Default values are applied where defined

Insert Multiple Rows in Oracle

```
INSERT ALL
  INTO tasks (title,
priority) VALUES ('Task
1', 1)
  INTO tasks (title,
priority) VALUES ('Task
2', 2)
  INTO tasks (title,
priority) VALUES ('Task
3', 3)
SELECT * FROM dual;
```

INSERT INTO SELECT (Oracle)

This statement inserts data from one table into another.

Syntax:

```
INSERT INTO table_name (column_list)
SELECT column_list
FROM another_table
WHERE condition;
```

Example

```
INSERT INTO tasks (title)
SELECT department
FROM employee;
```

Insert selected rows:


```
INSERT INTO tasks (title)
SELECT email
FROM employee
WHERE empNo = 'E101';
```

Using SELECT Inside VALUES

```
INSERT INTO tasks
(task_id, title, priority)
VALUES (
    (SELECT COUNT(*)
     FROM employee),
    (SELECT email FROM
     employee WHERE empNo =
     'E102'),
    (SELECT COUNT(*)
     FROM employee)
);
```

UPDATE Statement

The UPDATE statement modifies existing rows in a table.

Syntax:

```
UPDATE table_name
SET column1 = value1,
    column2 = value2
WHERE condition;
```

Update Single Column

```
UPDATE employee
SET empName =
'BlackSmith'
WHERE empNo = 'E100';
```

Update Multiple Columns

```
UPDATE employee
SET empName =
'BlackSmith',
    department = 'Iron'
WHERE empNo = 'E100';
```

Update Using REPLACE Function

```
UPDATE employee
SET email =
REPLACE(email,
'@colorado.edu',
'@nu.edu.pk')
WHERE empNo = 'E100';
```

DELETE Statement

The DELETE statement removes rows from a table.

Syntax:

```
DELETE FROM table_name
WHERE condition;
```

Delete Specific Row

```
DELETE FROM  
employee  
WHERE empNo = 'E101';
```

Delete All Rows

```
DELETE FROM  
employee;
```

TRUNCATE TABLE

To remove all rows quickly:

```
TRUNCATE TABLE  
employee;
```

Note: TRUNCATE is faster than DELETE because it does not generate undo for each row.

Tasks:

1. Write a SQL statement to create a tasks table with the following columns:
 - i) task_id (number, primary key)
 - ii) title (VARCHAR2(255), NOT NULL, UNIQUE)
 - iii) name (VARCHAR2(100))
 - iv) start_date (DATE)
 - v) due_date (DATE)
 - vi) priority (NUMBER(1), default 3, NOT NULL)
 - vii) description (CLOB)
2. Insert three tasks into the tasks table using a single INSERT ALL statement with the following details:
 - Task 1: Database Design, name = Ali Khan, priority = 1
 - Task 2: API Development, name = Sara khan, priority = 2
 - Task 3: Frontend UI, name = Hamid Ali, priority = 3
3. Insert a Task Using Employee Data
Insert a new task titled Prepare HR Report with priority 2, but use the employee name from the employees table where employee_id = 101 as the name column value.
4. Insert a Task with Partial Data
Insert a task titled Team Meeting with only the description column filled. Let the other columns use default values or remain NULL where applicable.
5. Write a SQL query to display all columns from the tasks table to verify all inserted data.
6. Retrieve the title and priority of all tasks where priority = 2.
7. Retrieve all tasks whose title contains the word 'Report'.
8. Display all tasks ordered by title in ascending order.
9. Display distinct priority values from the tasks table.
10. Update the priority of the task with task_id = 2 to 1.
11. Change the title of the task with task_id = 5 to Update Employee Data.
12. Increase the priority of all tasks with priority = 3 to 2.
13. Count the total number of tasks for each priority value.
14. Find the average priority of all tasks.
15. Find the maximum priority among all tasks.
16. Count the number of tasks whose title contains 'Report' for each priority.
17. Find the total number of tasks whose priority is greater than 1.
18. Find priority levels with more than 2 tasks.
19. Display all tasks ordered by title ascending.

20. Display tasks ordered by priority descending and then title ascending.
21. Display tasks ordered by priority ascending.
22. Copy all tasks from tasks_archive table to tasks.
23. Copy only tasks with priority = 2 from tasks_archive to tasks.
24. Insert a task into tasks using VALUES (SELECT ...) to copy the name of the employee with employee_id = 101 from the employees table.
25. Truncate the tasks table to remove all data.